



# Functional Product Requirements

## Agentic Travel Planning Platform

*B2B platform · Orchestrator + specialist agents · Headless / bring-your-own-agent*

| Field    | Detail                                          |
|----------|-------------------------------------------------|
| Product  | Smartinerary — Agentic Travel Planning Platform |
| Document | Functional PRD                                  |
| Version  | Draft 1.0                                       |
| Status   | For review                                      |
| Audience | Product, Engineering, Partnerships, Design      |
| Date     | June 2026                                       |

## Contents

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Product Overview .....                              | 3  |
| 1.1 Problem Statement .....                            | 3  |
| 1.2 Goals and Non-Goals .....                          | 3  |
| 1.3 Definitions .....                                  | 4  |
| 2. Users and Personas .....                            | 4  |
| 3. Scope and Delivery Models .....                     | 4  |
| 4. Functional Requirements .....                       | 5  |
| 4.1 Intent and Intake .....                            | 5  |
| 4.2 Orchestration and Planning .....                   | 6  |
| 4.3 Specialist Agents .....                            | 6  |
| 4.4 Tool Boundary, Gating, and Human-in-the-Loop ..... | 6  |
| 4.5 Headless and Bring-Your-Own-Agent .....            | 7  |
| 4.6 Plan Presentation and Handoff .....                | 7  |
| 4.7 Event Backbone and State .....                     | 7  |
| 5. End-to-End Plan Flow .....                          | 8  |
| 6. Agent-to-Agent Communication Contract .....         | 8  |
| 6.1 Request envelope (orchestrator → specialist) ..... | 8  |
| 6.2 Reply envelope (specialist → orchestrator) .....   | 9  |
| 7. Service Data Model .....                            | 9  |
| 7.1 Requested entities .....                           | 9  |
| 7.2 Configured entities .....                          | 9  |
| 8. Provider Integration .....                          | 10 |
| 9. Deployment and Request Flow .....                   | 10 |
| 10. Non-Functional Requirements .....                  | 11 |
| 10.1 Performance .....                                 | 11 |
| 10.2 Security and Compliance .....                     | 11 |
| 10.3 Cost Control .....                                | 11 |
| 10.4 Reliability and Observability .....               | 11 |
| 11. Acceptance Criteria .....                          | 11 |
| 12. Open Questions and Assumptions .....               | 12 |

# 1. Product Overview

---

Smartinerary is a **business-to-business (B2B) agentic travel-planning platform**. It takes a natural-language travel goal together with hard constraints (dates, budget, purpose, corporate policy) and turns it into a priced, policy-compliant, bookable itinerary. Planning is performed by an orchestrator that decomposes the goal and fans work out to parallel specialist agents — flight, lodging, activity, ground transit, and policy — each of which searches and verifies against external providers and returns ranked, traceable candidates.

Smartinerary is sold to organizations, not to individual consumers. Buyers are travel-management companies, corporate travel programs, online travel agencies, and software vendors that embed travel planning into their own products. The platform is delivered three ways: a hosted web and mobile experience, versioned REST APIs, MCPs and a **headless integration surface** that lets a customer drive Smartinerary directly from their own AI assistant — Claude, Gemini, ChatGPT, or any client that speaks the Model Context Protocol (MCP) or the agent-to-agent (A2A) contract.

## 1.1 Problem Statement

Planning a compliant, multi-segment trip is slow and fragmented. Search, verification, policy checking, and reconciliation happen across many provider systems with inconsistent data and pricing that goes stale quickly. Existing tools either over-rely on a single large model (unpredictable cost, weak auditability) or are rigid deterministic flows that cannot reason over open-ended goals. Organizations need a platform that is fast and parallel, keeps a human in the loop for booking, produces a full audit trail, and can be embedded into whatever surface the customer already uses.

## 1.2 Goals and Non-Goals

### Goals

- Convert a natural-language travel goal plus constraints into a versioned, priced, policy-compliant itinerary that a human can confirm and book.
- Run independent specialist searches in parallel to keep end-to-end planning latency low.
- Keep AI confined to narrow, gated surfaces; route every external action through a single auditable tool boundary.
- Offer the same planning capability through a hosted UI, REST APIs, and a headless MCP/A2A surface a customer can call from their own agent.
- Enforce per-organization policy, budget, and consent on every plan.

### Non-Goals

- Smartinerary is not the system of record for booking or payment. It produces handoff links and structured segments; the booking transaction completes in the provider or the customer's booking engine.
- No direct-to-consumer offering in this release. All access is mediated by a customer organization.
- The platform does not autonomously book without human or programmatic confirmation; final commit is always an explicit step.

## 1.3 Definitions

| Term             | Meaning                                                                                                     |
|------------------|-------------------------------------------------------------------------------------------------------------|
| Orchestrator     | Owns the PlanGraph, the agent loop, and the merge step; decomposes a goal and dispatches subtasks.          |
| Specialist agent | A domain agent (flight, lodging, activity, ground, policy) that searches and verifies against providers.    |
| A2A contract     | The request/reply envelope used between orchestrator and specialists — one task equals one specialist call. |
| Tool boundary    | The single gated layer through which any external or context access passes; allowlisted and audited.        |
| Headless mode    | A customer driving the platform from their own AI client (Claude, Gemini, ChatGPT) over MCP / A2A.          |
| PriceSnapshot    | A time-keyed, TTL-bound record of provider pricing that every candidate option traces back to.              |
| Handoff          | The transition from a verified plan segment to the external system where booking completes.                 |

## 2. Users and Personas

Smartinerary serves organizations. Within a customer, the platform recognizes the following actors:

| Persona                | Context                         | Primary needs                                                                 |
|------------------------|---------------------------------|-------------------------------------------------------------------------------|
| Travel manager / agent | TMC or corporate travel desk    | Plan trips on behalf of travelers, enforce policy, reconcile options quickly. |
| Corporate traveler     | Employee of a customer org      | Submit a goal, review compliant options, confirm segments.                    |
| Corporate approver     | Budget or policy owner          | Review and approve plans that exceed thresholds or trigger policy rules.      |
| Integration developer  | Customer or partner engineering | Embed planning via REST or drive it headlessly from their own agent.          |
| Platform operator      | Smartinerary internal           | Configure agents, monitor cost and traces, manage providers and rollouts.     |

The policy and compliance agent runs in **corporate mode only**; leisure or unmanaged contexts skip policy gating but retain budget and consent checks.

## 3. Scope and Delivery Models

The same planning core is exposed through three delivery models. A customer may use one or combine them.

### 3.1 Hosted experience

Web and mobile apps for travel managers and travelers. Covers goal intake, plan review, option comparison, approval routing, and confirmation. Suited to customers without engineering resources to integrate.

### 3.2 REST APIs

Versioned, standards-based endpoints behind an external API gateway. Customers call planning, plan-status, itinerary, and handoff endpoints from their own backend to embed Smartinerary inside an existing product.

### 3.3 Headless / bring-your-own-agent

Smartinerary exposes its capabilities as tools over the **Model Context Protocol (MCP)** and as an **A2A endpoint**. A customer connects their own AI assistant — for example Claude, Gemini, or ChatGPT — and drives planning conversationally from inside that assistant, with no Smartinerary UI in the loop. The customer's agent becomes a client of the orchestrator; Smartinerary remains the authority for verification, policy, pricing, and audit.

**Why this matters.** Many customers have already standardized on an AI assistant for their teams. Headless mode meets them there: the planning intelligence, provider connectivity, and compliance live in Smartinerary, while the conversational surface stays in the tool the customer already trusts.

**Boundary in headless mode.** Regardless of which external model calls in, every provider search, verification, and handoff still passes through the same tool boundary with the same allowlist, human-in-the-loop gates, budget enforcement, and audit logging. The external model never reaches providers or context directly.

### 3.4 Out of scope (this release)

- Payment capture and settlement.
- Direct consumer sign-up outside a customer organization.
- Fully autonomous booking without a confirmation step.

## 4. Functional Requirements

Requirements are grouped by capability area and identified as FR-**<AREA>**-**<n>**. Each is independently testable. Priority: **M** = must-have for launch, **S** = should-have, **C** = could-have.

### 4.1 Intent and Intake

| ID       | Requirement                                                                                                                    | Pri |
|----------|--------------------------------------------------------------------------------------------------------------------------------|-----|
| FR-INT-1 | Accept a natural-language travel goal together with structured constraints (dates, budget, purpose, traveler).                 | M   |
| FR-INT-2 | Parse intent into a Trip object with a typed Constraints set; validate completeness before planning starts.                    | M   |
| FR-INT-3 | Prompt for missing or ambiguous required fields rather than guessing them.                                                     | M   |
| FR-INT-4 | Accept the same intent payload from the hosted UI, REST, and the headless MCP/A2A surface, producing an identical Trip object. | M   |
| FR-INT-5 | Support multi-currency budgets, normalizing through the currency service.                                                      | S   |

## 4.2 Orchestration and Planning

| ID       | Requirement                                                                                                                | Pri |
|----------|----------------------------------------------------------------------------------------------------------------------------|-----|
| FR-ORC-1 | Decompose a validated Trip into independent specialist subtasks and dispatch them in parallel.                             | M   |
| FR-ORC-2 | Issue each subtask as a single A2A request envelope (task_id, goal, constraints, context_ref, deadline, budget_remaining). | M   |
| FR-ORC-3 | Pass a pointer to shared state (context_ref), never a full context dump, between orchestrator and specialists.             | M   |
| FR-ORC-4 | Merge and reconcile verified candidates into a single coherent itinerary, resolving cross-segment feasibility.             | M   |
| FR-ORC-5 | Enforce a per-plan latency deadline and per-subtask budget; degrade gracefully to partial results when exceeded.           | M   |
| FR-ORC-6 | Maintain a PlanGraph capturing the decomposition, dispatch, and merge so any plan can be reconstructed.                    | S   |

## 4.3 Specialist Agents

| ID       | Requirement                                                                                                       | Pri |
|----------|-------------------------------------------------------------------------------------------------------------------|-----|
| FR-SPC-1 | Provide flight, lodging, activity, and ground-transit specialists that search and verify against their providers. | M   |
| FR-SPC-2 | Provide a policy and compliance specialist that runs in corporate mode and flags or blocks non-compliant options. | M   |
| FR-SPC-3 | Return a structured A2A reply envelope: status, ranked candidates[], assumptions[], cost, confidence, provenance. | M   |
| FR-SPC-4 | Trace every candidate to one or more PriceSnapshots via provenance; no option is returned without a snapshot.     | M   |
| FR-SPC-5 | Run with isolated context per specialist so one specialist cannot read another's working context.                 | M   |
| FR-SPC-6 | Allow providers behind a specialist to evolve or be swapped without changing the A2A contract.                    | S   |

## 4.4 Tool Boundary, Gating, and Human-in-the-Loop

| ID       | Requirement                                                                                                 | Pri |
|----------|-------------------------------------------------------------------------------------------------------------|-----|
| FR-GAT-1 | Route every external or context access through a single tool registry that enforces an allowlist per agent. | M   |
| FR-GAT-2 | Reject any tool or query an agent is not explicitly allowed to call.                                        | M   |
| FR-GAT-3 | Require a human-in-the-loop (HITL) gate before any irreversible or booking-adjacent action.                 | M   |
| FR-GAT-4 | Log every tool call with agent, arguments, result size, cost, and outcome to the tool-call log.             | M   |

| ID       | Requirement                                                                                              | Pri |
|----------|----------------------------------------------------------------------------------------------------------|-----|
| FR-GAT-5 | Apply the same boundary, allowlist, HITL gates, and audit when the caller is an external headless agent. | M   |
| FR-GAT-6 | Clamp numeric tool parameters to configured maximums to prevent resource exhaustion.                     | S   |

#### 4.5 Headless and Bring-Your-Own-Agent

| ID       | Requirement                                                                                                              | Pri |
|----------|--------------------------------------------------------------------------------------------------------------------------|-----|
| FR-HDL-1 | Expose planning capabilities as MCP tools discoverable and callable by external clients (e.g., Claude, Gemini, ChatGPT). | M   |
| FR-HDL-2 | Provide an A2A endpoint so a customer's orchestrating agent can dispatch tasks using the standard envelope.              | M   |
| FR-HDL-3 | Authenticate each headless connection to a customer organization and scope all access to that tenant.                    | M   |
| FR-HDL-4 | Return structured, model-agnostic responses (Trip, candidates, itinerary, provenance) usable by any client model.        | M   |
| FR-HDL-5 | Enforce per-organization budget and token allowances on headless sessions, surfacing remaining budget to the caller.     | M   |
| FR-HDL-6 | Stream plan progress events to the headless client so a conversational assistant can narrate status in real time.        | S   |
| FR-HDL-7 | Let an operator enable, disable, and scope which capabilities are exposed headlessly per customer.                       | S   |

#### 4.6 Plan Presentation and Handoff

| ID       | Requirement                                                                                              | Pri |
|----------|----------------------------------------------------------------------------------------------------------|-----|
| FR-PRS-1 | Store the final itinerary as a versioned object with per-segment handoff links.                          | M   |
| FR-PRS-2 | Require explicit human or programmatic confirmation before any segment is handed off for booking.        | M   |
| FR-PRS-3 | Present ranked options with assumptions, price, and provenance visible to the reviewer.                  | M   |
| FR-PRS-4 | Route plans that breach budget or policy thresholds to an approver before confirmation (corporate mode). | M   |
| FR-PRS-5 | Emit notifications (push, in-app, email) on plan ready, approval needed, and handoff complete.           | S   |

#### 4.7 Event Backbone and State

| ID       | Requirement                                                                                           | Pri |
|----------|-------------------------------------------------------------------------------------------------------|-----|
| FR-EVT-1 | Publish session.state, plan.events, and provenance to an event backbone consumed across the platform. | M   |

| ID       | Requirement                                                                                             | Pri |
|----------|---------------------------------------------------------------------------------------------------------|-----|
| FR-EVT-2 | Persist itinerary state and PriceSnapshots so a plan survives client disconnect and can be resumed.     | M   |
| FR-EVT-3 | Maintain a shared backplane (itinerary state, PriceSnapshots, memory) that agents reference by pointer. | M   |
| FR-EVT-4 | Provide a TTL-keyed inventory cache for provider snapshots to reduce duplicate provider calls.          | S   |

## 5. End-to-End Plan Flow

A plan moves through six stages. The flow is identical whether the trigger comes from the hosted UI, a REST call, or a headless agent.

1. **Intent.** A natural-language goal arrives with dates, budget, and purpose.
2. **Intake.** The intake parser extracts and validates, producing a Trip with typed constraints.
3. **Event backbone.** Session state, plan events, and provenance are published for the rest of the platform.
4. **Plan and dispatch.** The orchestrator decomposes the goal and fans out subtasks to specialists in parallel.
5. **Specialist inference.** Specialists search, verify, enrich, and log; each returns verified candidates with provenance.
6. **Present and handoff.** The merged, priced, compliant itinerary is stored with handoff links; a human confirms and books each segment.

**Lookup and reference services.** Throughout, specialists call shared services for context and to log activity: the inventory cache (TTL-keyed snapshots), geo and routing (transfer feasibility), the currency service (multi-currency budgets), the memory / vector store (preference retrieval), the prompt and model registry (versioned prompts and models), and the tool-call log (audit and provenance).

## 6. Agent-to-Agent Communication Contract

Orchestrator and specialists communicate through a fixed request/reply contract. One task equals one specialist call; tasks are idempotent.

### 6.1 Request envelope (orchestrator → specialist)

| Field         | Meaning                                       |
|---------------|-----------------------------------------------|
| task_id       | Idempotent unit of work.                      |
| goal          | What to satisfy, expressed in scope terms.    |
| constraints[] | Hard rules that bound the search.             |
| context_ref   | Pointer to shared state — not a context dump. |



| Field            | Meaning                          |
|------------------|----------------------------------|
| deadline         | Latency budget for this subtask. |
| budget_remaining | Spend and token allowance.       |

## 6.2 Reply envelope (specialist → orchestrator)

| Field         | Meaning                                 |
|---------------|-----------------------------------------|
| status        | done · partial · blocked · needs input. |
| candidates[]  | Ranked, verified options.               |
| assumptions[] | What was inferred to proceed.           |
| cost          | Tokens and tool calls consumed.         |
| confidence    | Specialist self-assessment.             |
| provenance    | Snapshot IDs behind each option.        |

## 7. Service Data Model

The platform is organized around a small set of core entities. The model separates what is **requested** (a traveler's plan) from what is **configured** (the roles, segment types, constraints, and gateways an operator sets up).

### 7.1 Requested entities

| Entity                      | Relationship                                                                  |
|-----------------------------|-------------------------------------------------------------------------------|
| Traveler                    | Creates and searches itineraries; holds preferences (plans, budget, loyalty). |
| Itinerary                   | Generated for a traveler; comprises ordered segments.                         |
| Segment                     | A bookable unit (flight, hotel, rail/car, dining) attached to a provider.     |
| Booking Handoff             | Routes a segment between the itinerary and a provider for booking.            |
| Provider / Provider Network | External source a segment is fulfilled against.                               |
| Consent Grant               | Attached to a traveler; governs data use and access.                          |
| Metrics & Audit             | Records activity and provenance across the plan lifecycle.                    |

### 7.2 Configured entities

| Entity           | Comprises                                            |
|------------------|------------------------------------------------------|
| Specialist Roles | Flight planner, hotel planner, activity planner, ... |
| Segment Types    | Flight, hotel, rail / car, dining.                   |
| Constraints      | Budget cap, time window, policy rule, ...            |

| Entity            | Comprises                                         |
|-------------------|---------------------------------------------------|
| Provider Gateways | GDS gateway, hotel gateway, activity gateway, ... |

## 8. Provider Integration

Specialists reach providers through provider adapters behind the service-provisioning layer. The platform mostly reads (search and verify); the only handoff out is the booking handoff. Flight connectivity spans legacy GDS and modern API-first sources:

| Provider       | Type                                    | Booking model                                                          |
|----------------|-----------------------------------------|------------------------------------------------------------------------|
| Amadeus        | Legacy GDS (+ self-service tier)        | Enterprise needs IATA/ARC; self-service is lighter.                    |
| Sabre          | Legacy GDS                              | Enterprise / accreditation.                                            |
| Travelport     | Legacy GDS (Galileo, Worldspan, Apollo) | Enterprise.                                                            |
| Duffel         | Modern API-first                        | Per-booking with volume discounts; cost-effective vs. traditional GDS. |
| Kiwi (Tequila) | Modern aggregator                       | Access now requires a project with high monthly active users.          |
| Skyscanner     | Metasearch / aggregator                 | Affiliate / comparison.                                                |

Hotel, activity, and ground/rail providers integrate through their own gateways under the same adapter pattern, so a provider can be added or swapped without changing the A2A contract or specialist logic.

## 9. Deployment and Request Flow

The platform runs behind a defined boundary. A typical request traverses edge protection, authentication, agent functions, and the supporting stores, with observability throughout.

1. Sign in — the user authenticates from the browser client.
2. Load UI — static assets are served via the CDN / static host.
3. Edge protect — the web application firewall screens traffic.
4. Route — the API gateway routes the request.
5. Authenticate — an authorizer function checks the token against the identity provider and policy store.
6. Invoke — agent functions run the backing logic.
7. Read use cases — the use-cases store supplies the configured planning behavior.
8. Provision — the IaC engine provisions stacks for deployed use cases.
9. Store config — LLM configuration is written to the config and artifact stores, keyed by stack ID.
10. Observe — activity flows to observability dashboards.

## 10. Non-Functional Requirements

---

### 10.1 Performance

- Parallel specialist execution so total plan latency tracks the slowest specialist, not the sum.
- TTL-keyed inventory caching to avoid redundant provider calls within a snapshot window.
- Streaming progress events so UI and headless clients see status without polling.

### 10.2 Security and Compliance

- Single tool boundary with per-agent allowlists; no agent reaches providers or context directly.
- Per-tenant isolation derived from the authenticated connection, never from client- or model-supplied identity.
- Consent grants and corporate policy enforced before handoff.
- Full audit trail: every tool call, snapshot, and plan decision is logged with provenance.

### 10.3 Cost Control

- Per-plan and per-subtask budget and token allowances, enforced before spend, with graceful partial results.
- Deterministic supervisor for routing rather than a reasoning model, reserving model spend for specialist inference.
- Versioned prompt and model registry so model selection and cost are controlled centrally.

### 10.4 Reliability and Observability

- Plans survive client disconnect; itinerary state and snapshots are persisted and resumable.
- Idempotent A2A tasks so a retried subtask does not double-spend or duplicate work.
- Custom observability dashboards over traces, cost, and plan outcomes.

## 11. Acceptance Criteria

---

The release is acceptable when the following hold across all three delivery models (hosted, REST, headless):

1. A natural-language goal with valid constraints produces a versioned, priced, policy-compliant itinerary with per-segment handoff links.
2. Specialist subtasks run in parallel and every returned candidate traces to at least one PriceSnapshot.
3. No external action occurs without passing the tool boundary, and no booking-adjacent action occurs without a confirmation gate.
4. A headless client (e.g., Claude, Gemini, or ChatGPT) can complete a full plan via MCP/A2A, scoped to its organization, under enforced budget.
5. In corporate mode, plans breaching budget or policy thresholds are routed for approval before confirmation.
6. Every tool call, plan decision, and snapshot is present in the audit trail with provenance.

## 12. Open Questions and Assumptions

---

- Which MCP-capable clients are certified at launch versus best-effort compatible?
  - Per-tenant rate and budget defaults for headless sessions, and how overage is surfaced to the customer's agent.
  - Depth of approval workflows in corporate mode (single vs. multi-step approval chains).
  - Provider accreditation requirements (e.g., IATA/ARC) gating which flight sources are live per customer.
  - Retention policy for PriceSnapshots and tool-call logs by tenant and region.
- 

*This functional PRD describes intended platform behavior derived from the Smartinerary reference architecture. Where implementation and this document diverge, reconcile and update this document.*